ASSIGNMENT - 19.1

2303A51945
B - 27

Task1–PythontoJavaConversion
Task:
TranslateasimplePythonclassintoJavausingAIassistance.
Instructions:
• PromptAItoconvertaPythonclasswithattributesand methods into equivalent Java code.
• Verifythatconstructors,methodsignatures,anddatatypes are properly handled.
• Runbothversions(Python&Java)tocompareexpected outputs.
ExpectedOutput:
• EquivalentworkingJavaclasstranslatedfromPython.

Pythoncode:

```
classEmployee:
    def __init__(self,name,salary,department):
        self.name = name
        self.salary = salary
        self.department=department

    defgive_raise(self,amount):
        self.salary += amount
        return self.salary
```

```
    defdisplay_info(self):
        returnf"Employee:{self.name},Salary:
 {self.salary},Department:{self.department}"

emp=Employee("Rohit",50000,"Engineering")
print(emp.display_info())
emp.give_raise(5000)
print(emp.display_info())
```

Output:

```
Employee:Rohit,Salary:50000,Department: Engineering
Employee:Rohit,Salary:55000,Department: Engineering
```

Prompt:
IhaveaPythonclassandIneedtoconvertitintoJava. Generate a fully working Java class with:

- properconstructor
- correctdatatypes
- equivalentattributesandmethods
- amain()methodtoruntheclass
- anoutputthatmatchesthePythonversion

HereisthePythoncode:
(classabove)

Convertitandshowtheexpectedoutput.

Code:

```
publicclassEmployee{
    private String name;
    private double salary;
    private String department;

    publicEmployee(Stringname,doublesalary, String
department) {
        this.name=name;this.salary =
        salary;
        this.department=department;
    }

    publicdoublegiveRaise(doubleamount){ this.salary
        += amount;
        returnthis.salary;
    }

    publicStringdisplayInfo(){
        return"Employee:"+name+",Salary:"+ salary +
", Department: "+ department;
    }

    public static void main(String[] args) {
        Employeeemp=newEmployee("Rohit",50000,
"Engineering");
        System.out.println(emp.displayInfo());
        emp.giveRaise(5000);
```

```
        System.out.println(emp.displayInfo());
    }
}
```

Output:

```
Employee:Rohit,Salary:50000.0,Department:
Engineering
Employee:Rohit,Salary:55000.0,Department:
Engineering
```

Task2–C++toPythonFunctionConversion
Task:
ConvertaC++functionforfactorialcalculationintoPython.
Instructions:
• AskAItotranslatearecursiveC++factorialfunctioninto
Python.
• Validatecorrectnessbytestingwithsampleinputs.
• EnsurePythonversionfollowsPythonicconventions.
Expected Output:
• AcorrectPythonimplementationequivalenttotheoriginal
C++ factorial function.

C++Code:

```
#include
<iostream>usingnames
pacestd;
```

```
    if(n<=1)
        return1;
    else
        returnn*factorial(n-1);
}

intmain(){
    cout<<"factorial(3)="<<factorial(3)<<endl;
    cout<<"factorial(5)="<<factorial(5)<<endl;
    cout<<"factorial(7)="<<factorial(7)<<endl;
    return0;
}
```

Output(C++):

```
factorial(3) = 6
factorial(5) = 120
factorial(7) = 5040
```

Prompt:
TranslatethefollowingrecursiveC++factorialfunctioninto Python.
MakesurethePythonversion:
• Usesrecursionliketheoriginal
• FollowsPythonicstyle
• Producesthesameoutput
AlsorunthePythonversionforinputs3,5,and7.

PythonCode:

```
deffactorial(n):
    if n <= 1:
        return1
    returnn*factorial(n-1)

print("factorial(3) =", factorial(3))
print("factorial(5) =", factorial(5))
print("factorial(7) =", factorial(7))
```

Output(Python):

```
factorial(3) = 6
factorial(5) = 120
factorial(7) = 5040
```

Task3–SQLtoPandasQuery Task:
TranslateaSQLqueryintoaPandasDataFrameoperationin Python.
Instructions:
• ProvideAIwithaSQLquery(e.g.,SELECTname,salary
FROM employees WHERE salary > 50000).
• AskAItogenerateequivalentPandascode.
• Testthegeneratedcodeonasampleda taFrame.
Expected Output:
• EquivalentPandasquerythatretrievesthecorrectsubsetof

data.

SQLCode:

```
SELECTname,salary
FROM employees
WHERESalary>50000;
```

Output(SQLExpectedResult):

```
name       salary
----------------------------
Rohit      72000
Meera      85000
```

Prompt:
ConvertthefollowingSQLqueryintoanequivalentPandas DataFrame operation.
Makesurethegeneratedcode:
• UsesasampleemployeesDataFrame
• Appliesthesamefilteringcondition
• Selectsthesamecolumns
• ProducesthesamefinaloutputastheSQLquery

SQLquery:
SELECTname,salaryFROMemployeesWHEREsalary>50000;

GeneratethePandasversionandshowtheoutput.

Python Code (Pandas Equivalent):

```python
importpandasaspd

#SampleDataFrame
data = {
    "name":["Rohit","Meera","Amit","Sara"],
    "salary": [72000, 85000, 45000, 30000]
}

df=pd.DataFrame(data)

# Pandas equivalent of the SQL query
result=df[df["salary"]>50000][["name",
"salary"]]

print(result)
```

Output(Pandas):

```
      namesalary
0  Rohit   72000
1  Meera   85000
```

Task4–Real-TimeApplication:Multi-LanguageSnippet Converter

Scenario:

Studentswillchooseareal-timeusecasewheretheyneedcode in multiplelanguages.Example:

• ConvertingaPythonwebscrapingsnippetintoJavaScript

forbrowserautomation.
• TranslatingaJavamethodintoC#forenterprise applications.
Instructions:
• PromptAItotranslatecodebetweentwochosenlanguages.
• Testbothimplementationstoensuretheyproducethesame results.
• Documentchallengesfacedduringtranslation(e.g.,syntax, libraries).
ExpectedOutput:
• Workingequivalentcodesnippetsinatleasttwodifferent programming languages.

PythonCode(OriginalSnippet):

```
importrequests
frombs4importBeautifulSoup

defget_title(url):
    response=requests.get(url)
    soup=BeautifulSoup(response.text,
"html.parser")
    returnsoup.title.string

print(get_title("https://www.python.org"))
```

Output(PythonExpectedResult):

```
WelcometoPython.org
```

Prompt:

Translate the following Python web-scraping snippet into an equivalentJavaScriptversionthatrunsinabrowserenvironment using fetch() and DOMParser.
Makesurebothimplementations:
• Extractthepagetitle
• Producethesameoutput
• Useappropriatelibrariesforeachlanguage
Afterconverting,showtheJavaScriptcodeanditsexpected output, followed by a short list of challenges faced during translation.

Pythonsnippet:
importrequests
frombs4importBeautifulSoup

defget_title(url):
    response=requests.get(url)
    soup=BeautifulSoup(response.text,"html.parser") return
    soup.title.string

JavaScriptCode(ConvertedVersion):

```
asyncfunctiongetTitle(url){
    const response = await fetch(url);
    consthtml=awaitresponse.text(); const
    parser = new DOMParser();
    constdoc=parser.parseFromString(html,
"text/html");
    returndoc.querySelector("title").innerText;
}
```

```
getTitle("https://www.python.org").then(title=>{
    console.log(title);
});
```

Output(JavaScriptExpectedResult):

```
WelcometoPython.org
```

TranslationChallenges(DocumentedNotes):

1. Python’srequests+BeautifulSoupworksynchronously,while JavaScript uses async fetch + DOMParser.
2. Exceptionhandlingdiffers(Python’stry/exceptvs.JS Promises).
3. BeautifulSouphasconvenientparsinghelpers;DOMParser requires precise HTML queries.
4. RunningJSscrapingrequiresabrowserenvironmentor additional Node.js libraries.
5. Differencesinindentation,typing,andlanguagesyntaxmustbe adapted manually.